1 Solution — GIAM 2.3 Problem 3

1.1 Problem

Draw pairs of related digital logic circuits that illustrate De Morgan's laws.

1.2 The Laws

De Morgan's two laws say that a negation may be pushed inside a conjunction or disjunction, provided the connective is flipped:

$$\neg(A \wedge B) \;\cong\; \neg A \vee \neg B, \qquad\qquad \neg(A \vee B) \;\cong\; \neg A \wedge \neg B.$$

Each law therefore corresponds to **two circuits that compute the same output** from the same inputs.

1.3 The Circuits

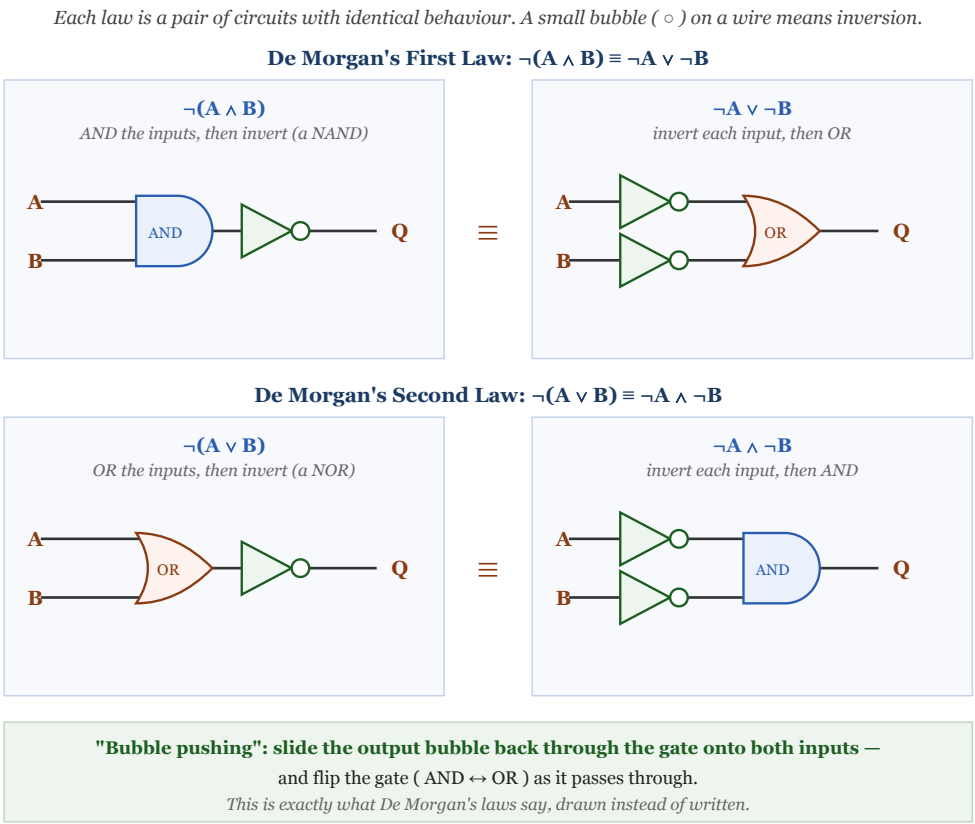

Figure 1.1: Two pairs of digital logic circuits. Top pair, De Morgan's first law: on the left, inputs A and B feed an AND gate whose output passes through a NOT gate to give Q — this computes not(A and B), a NAND. On the right, A and B each pass through their own NOT gate and the two inverted signals feed an OR gate to give Q — this computes not-A or not-B. The two circuits are marked equivalent. Bottom pair, De Morgan's second law: on the left, A and B feed an OR gate whose output passes through a NOT gate — this computes not(A or B), a NOR. On the right, A and B each pass through a NOT gate and the inverted signals feed an AND gate — this computes not-A and not-B. Again the two are marked equivalent. A closing note explains bubble pushing: slide the output bubble back through the gate onto both inputs, and flip the gate from AND to OR or OR to AND as it passes through.

Top pair (first law). On the left, A and B are combined by an AND gate and the result is inverted — a **NAND**. On the right, each input is inverted first and the results are combined by an OR gate. Both produce the same Q.

Bottom pair (second law). On the left, A and B are combined by an OR gate and inverted — a **NOR**. On the right, each input is inverted first and the results are ANDed. Again both produce the same Q.

1.4 Verification

Tracing all four input combinations through each circuit confirms the pairs agree:

A	B	$\neg(A \wedge B)$	$\neg A \vee \neg B$	$\neg(A \vee B)$	$\neg A \wedge \neg B$
T	T	**F**	**F**	**F**	**F**
T	F	**T**	**T**	**F**	**F**
F	T	**T**	**T**	**F**	**F**
F	F	**T**	**T**	**T**	**T**

Table 1.1

Columns 3 and 4 match (first law); columns 5 and 6 match (second law). ■

1.5 Remark — "Bubble Pushing": The Visual Form of the Laws

Engineers draw inversion as a small **bubble** ($\circ$) rather than a full triangle, and in that notation De Morgan's laws become a purely graphical manipulation called **bubble pushing**:

> *Slide the bubble from the output back through the gate onto both inputs — and change the gate's shape as it passes through (*$\text{AND} \leftrightarrow \text{OR}$*).*

A NAND (AND with an output bubble) becomes an OR with two input bubbles; a NOR becomes an AND with two input bubbles. The rule "move the bubbles, flip the gate" is exactly $\neg(A \wedge B) = \neg A \vee \neg B$ restated in pictures. Because the manipulation is visual, designers apply it fluently while reading a schematic — no algebra required.

1.6 Remark — Why the Connective *Must* Flip

The flip is not a convention but a necessity, and one row of the table shows why. Take A true and B false. Then:

- $\neg(A \wedge B)$ is **true** (the AND fails, so its negation holds);
- $\neg A \vee \neg B$ is **true** (because $\neg B$ holds) ✓ — matches;
- but $\neg A \wedge \neg B$ would be **false** ($\neg A$ fails).

So negating "both" gives "at least one fails," not "both fail." In English: the opposite of *"both switches are on"* is *"at least one switch is off,"* not *"both switches are off."* Forgetting the flip — writing $\neg(A \wedge B) = \neg A \wedge \neg B$ — is one of the most common errors in elementary logic, and this row is the counterexample that kills it.

1.7 Remark — This Is Why NAND and NOR Are Universal

The circuits above quietly explain the functional completeness proved in Problem 2.1.5. Because a NAND gate is *also* an OR gate with inverted inputs, a single NAND can play either role depending on how you feed it. Combined with the self-negation trick $\neg X = X \mid X$ (tie both inputs together), one gate type yields NOT, AND, and OR — hence everything. De Morgan's laws are the bridge: they are what allow a chip full of identical NAND gates to realise both conjunctive and disjunctive logic. A fabricator masters one gate; De Morgan supplies the rest.

1.8 Remark — Redrawing a Schematic Without Changing It

A practical use of these pairs is that they let a designer redraw a circuit to make its *intent* legible without altering its behaviour. Suppose a safety interlock should fire when "the door is not closed **or** the guard is not engaged." Written directly that is $\neg D \vee \neg G$ — two inverters and an OR. By the first law it is identical to $\neg(D \wedge G)$: a single NAND fed by both sensors, reading naturally as "*not* (door closed **and** guard engaged)." Same silicon behaviour, fewer gates, and a description that matches how an engineer would state the requirement. Choosing between logically equivalent forms for clarity or cost is precisely the payoff of knowing the laws (as in Problem 2).

1.9 Remark — The Laws Are Dual to One Another

The two laws are not independent facts but mirror images under the **duality** that swaps $\wedge \leftrightarrow \vee$ and $\mathbf{T} \leftrightarrow \mathbf{F}$ throughout. Applying that swap to the first law produces the second, and vice versa — which is visible in the figure as the top and bottom rows being the same picture with the gate shapes exchanged. Duality runs through all of Boolean algebra: absorption came in a dual pair in Problem 2 ($A \vee (A \wedge B) = A$ and $A \wedge (A \vee B) = A$), as did the distributive laws in Problem 1. Prove one member of a dual pair and the other follows for free, which is why these identities are conventionally tabulated two at a time.